FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Computer System and Architecture

Machine Problem

MIDTERM

Aaron Gabriel M. Pahutan	Sir. Hadji Tejuco
Name of Student	Name of Professor
10 / 10 / 26	10 / 10 / 26
Data Performed	Date Submitted

List the steps that you perform to come up with the program.

Linux System Calls (int 0x80):

- Assembly programs interact with the operating system kernel via system calls.
- The system call number is loaded into the EAX register (e.g., 4 for sys_write, 1 for sys_exit).
- Arguments are passed consecutively into registers such as EBX (file descriptor, e.g., 1 for STDOUT), ECX (pointer to the message/data buffer), and EDX (length of the data). Triggering int 0x80 executes the kernel routine.

Data Definition Directives (DB) and Constants (EQU):

- The DB (Define Byte) directive is used in the data segment to allocate and initialize character strings, ASCII art, and row/column layout borders.
- Special bytes like 0xA (line feed) and 0xD (carriage return) are embedded to format output rows properly.
- The EQU directive defines constant values like system call identifiers and file descriptors for cleaner code maintenance.

Location Counter Symbol ($):

- The location counter $ represents the current address during assembly. By defining len equ $ - msg, the assembler automatically calculates the exact byte length of our formatted slambook sections and ASCII art strings without manual counting.

Row/Column Formatting & ASCII Art:

- To improve the appearance of the slambook (inspired by the graphical layout in the reference image), we utilize custom borders, spaced column alignments, and an ASCII art header rendered directly to STDOUT via sys_write.

Paste the source code and explain the functionalities and take a screenshot of the final output.

```
;
==========================================================================
======
; Program Name: Slambook Application with ASCII Art & Column Formatting
; Description: Displays an enhanced ASCII art header and a structured
;              slambook form using Linux system calls and data directives.
;
==========================================================================
======
```

```
SYS_EXIT    equ 1       ; System call number for exit[cite: 51]
SYS_WRITE   equ 4       ; System call number for write[cite: 27, 51]
STDOUT      equ 1       ; File descriptor for standard output (screen)

section .data
    ;
==========================================================================
==
    ; SECTION 1: ASCII ART HEADER
    ; Uses DB (Define Byte) to store custom text graphics and line breaks (0xA, 0xD)
    ;
==========================================================================
==
    ascii_art db "  _____ _             _                 _    ", 0xA, 0xD
              db " / ____| |           | |               | |   ", 0xA, 0xD
              db "| (___ | |__  _ _ __ | |__   ___   ___ | | __", 0xA, 0xD
              db " \___ \| |/ _` | '__| '_ \ / _ \ / _ \| |/ /", 0xA, 0xD
              db " ____) | | (_| | |  | |_) | (_) | (_) |   < ", 0xA, 0xD
              db "|_____/|_|\__,_|_|  |_.__/ \___/ \___/|_|\_\", 0xA, 0xD
              db "=============================================", 0xA,
0xD, 0
    len_art   equ $ - ascii_art   ; Uses location counter ($) to calculate length

    ;
==========================================================================
==
    ; SECTION 2: ABOUT ME INFORMATION
    ; Stores personal profile details aligned in a row/column format
    ;
==========================================================================
==
    section_1 db "[ ABOUT ME SECTION ]", 0xA, 0xD
              db "-----------------------------------------", 0xA, 0xD
              db " Name   : [ Aaron Pahutan ]               ", 0xA, 0xD
              db " Email  : [ ampahutan@fit.edu.ph ]        ", 0xA, 0xD
              db " Motto  : [ Code with logic and precision ]  ", 0xA, 0xD
              db "-----------------------------------------", 0xA, 0xD, 0
    len_sec1  equ $ - section_1   ; Dynamic length calculation for Section 1[cite: 108]

    ;
==========================================================================
==
    ; SECTION 3: FAVES & HOBBIES
```

```
    ; Stores favorite media and personal interests with custom borders
    ;
================================================================================
    section_2 db "[ MY FAVES & HOBBIES ]", 0xA, 0xD
              db "-----------------------------------------", 0xA, 0xD
              db " Color  : [ Crimson Red ]                 ", 0xA, 0xD
              db " Hobbies: [ Gaming & Cycling ]              ", 0xA, 0xD
              db " Fave TV: [ Invincible ]                  ", 0xA, 0xD
              db "-----------------------------------------", 0xA, 0xD, 0
    len_sec2  equ $ - section_2   ; Dynamic length calculation for Section 2[cite: 108]

section .text
    global _start       ; Entry point declaration for the linker (ld)

_start:
    ;
================================================================================
    ; PRESENTATION BLOCK A: Printing the ASCII Art Header
    ; Loads sys_write (4) into EAX, STDOUT (1) into EBX, and string pointer into
ECX[cite: 26, 51]
    ;
================================================================================
    mov eax, SYS_WRITE
    mov ebx, STDOUT
    mov ecx, ascii_art
    mov edx, len_art
    int 0x80            ; Triggers Linux kernel interrupt to display header

    ;
================================================================================
    ; PRESENTATION BLOCK B: Printing the "About Me" Section
    ; Outputs Aaron Pahutan's name and contact information[cite: 26]
    ;
================================================================================
    mov eax, SYS_WRITE
    mov ebx, STDOUT
    mov ecx, section_1
    mov edx, len_sec1
```

```
    int 0x80

    ;
=================================================================
==
    ; PRESENTATION BLOCK C: Printing "My Faves & Hobbies" Section
    ; Outputs hobbies (Gaming & Cycling) and favorite show (Invincible)[cite: 26]
    ;
=================================================================
==
    mov eax, SYS_WRITE
    mov ebx, STDOUT
    mov ecx, section_2
    mov edx, len_sec2
    int 0x80

    ;
=================================================================
==
    ; PRESENTATION BLOCK D: Program Termination
    ; Calls sys_exit (1) with status code 0 to return control to OS
    ;
=================================================================
==
    mov eax, SYS_EXIT
    xor ebx, ebx
    int 0x80
```